11

Quick Reference Cheat Sheet for MSD Interview

This appendix provides a helpful reference list of topics to consider during your MSD interviews. They're organized by domain, making them easier to navigate.

Network

Category	Key considerations
Core protocols	• **Communication protocols**: REST, GraphQL, protocol buffers, or gRPC. • **Real-time updates**: HTTP polling, SSE, WebSockets. • **Request management**: idempotency keys, priority queuing, rate limiting, request throttling.
Data flow patterns	• **Pagination**: offset vs. cursor-based. • **Offline capabilities**: offline-first design, optimistic UI updates, conflict resolution. • **Network resilience**: connectivity handling, exponential backoff.
Security	• **Authentication and authorization**: token lifecycle, biometric auth, recovery flows. • **Security infrastructure**: encryption in transit and at rest, certificate pinning, compliance requirements.

Data management

Category	Key considerations
Architecture	• **Core data flow**: patterns for data flow, layer responsibilities, error handling, error propagation.
Storage and caching	• **Storage options**: key-value, relational/non-relational DB, custom/binary stores, secure storage. • **Caching**: in-memory vs. disk trade-offs. • **Eviction policies**: TTL, size-based eviction, priority-based eviction. • **Privacy**: PII handling, GDPR compliance, data retention policies. • **Synchronization**: sync strategies, conflict resolution, delta sync.
Data performance	• **Pre-fetching**: predictive fetching, resource usage trade-offs, prioritize critical paths. • **Background processing**: task efficiency, battery impact, OS background execution limits.
UI & Interactions	• **UI states**: loading, empty, error, content. • **Input handling**: validation on client and server side, sanitization, feedback for errors. • **Search**: local vs. server-side search, indexing, typo tolerance.

Feature development

Category	Key considerations
Version management	• **Force upgrading**: soft vs. hard upgrades, when to require updates, graceful prompts, version compatibility policies. • **Safe rollout**: phased rollouts, feature flags, rollback procedures. • **CI/CD**: automate build, test and deployments, code quality and security checks, ensure reproducible builds.
Dynamic configuration	• **Remote config**: feature control without app updates, default values for offline scenarios. • **A/B testing**: experiments with clear metrics, user segmentation. • **Analytics**: performance and business metrics, crash monitoring, logging.
Code architecture	• **Modularization**: separation of concerns, clear interfaces, balance granularity with practical maintenance. • **Dependency injection**: testability, object lifecycle management, minimize global state and singletons. • **Third-party libraries**: security and maintenance evaluation, size impact.
User experience	• **Localization**: text expansion/contraction, cultural and regional differences. • **Accessibility**: screen readers, contrast, touch targets, test with accessibility tools. • **Crash reporting**: automated collection, issue prioritization by impact and frequency, ensure right PII handling.

Performance

Category	Key considerations
User experience	• **Pre-fetching**: predictive fetching, resource usage trade-offs, prioritize critical paths. • **App startup**: optimize initialization, defer non-critical tasks, measure cold/warm start times. • **Stability**: prevent crashes, implement error boundaries.
Resource management	• **Battery & CPU**: minimize background processing, batch operations. • **Network efficiency**: compress payloads, minimize requests, adapt quality on connection type. • **App size**: optimize assets, use app bundles, remove unused code.
Technical optimizations	• **Caching strategy**: cache appropriate responses, invalidation policies, multi-level caching. • **Lazy loading**: on-demand components and data loading, defer heavy processing until needed, prioritize visible content first. • **Concurrency**: proper threading models, avoid blocking main thread, manage thread contention and race conditions. • **Hardware acceleration**: GPU usage for animations and rendering, specific hardware optimizations.
Monitoring	• **Observability**: performance metrics, proactive alerting, traces for complex operations. • **Business metrics**: connect performance to business outcomes, track user engagement, user funnels, identify bottlenecks affecting revenue.

Team & Organization

infrastructure	• **Design system**: consistent visual language, reusable components. • **Development efficiency**: optimized builds, coding standards, effective code review processes.
Process & quality	• **Code quality**: static analysis, automated testing, technical debt. • **Risk management**: early identification, contingency plans.
Team & culture	• **Developer experience**: efficient workflows, documentation. • **Team growth**: onboarding, knowledge sharing, continuous learning.
Company context	• **Available resources**: infra constraints, team size and expertise, budget, in-house development vs. 3rd party investments. • **Business priorities**: objectives alignment, target markets, monetization strategies, short-term needs vs. long-term platform health. • **Outage handling**: escalation procedures, recovery playbooks, appropriate redundancies. • **Tech stack**: integration with existing systems, platform consistency, future ecosystem evolution, standardization vs. specialized tools.

📌 **Remember:** This list isn't meant to be a checkbox exercise where you cover everything. Strategic selection is key! Focus on the considerations most relevant to your specific design challenge, and be ready to explain why those topics are important. Your ability to make thoughtful trade-offs based on the system's unique requirements demonstrates stronger design thinking than attempting to cover every possible topic.

Mark as Complete